

Lista de Exercícios — Pilhas (C)

Estrutura base (vista em aula):

```
#define MAX_ELEMENTOS 10

struct pilha {
    int elementos[MAX_ELEMENTOS];
    int topo;
};
```

Operações já implementadas em aula (você pode reutilizá-las): `cria`, `empilha`, `desempilha`, `tamanho`, `destrói`, além das versões que tratam pilha cheia e pilha vazia.

Regra geral da lista: salvo indicação em contrário, você não pode acessar o vetor `elementos` nem o campo `topo` diretamente fora das funções da própria pilha. Use apenas as operações da interface (`empilha`, `desempilha`, `tamanho`, etc.). Isso é justamente o que torna a pilha uma estrutura de dados e não "só um vetor".

Nível 1 — Aquecendo com a interface

1.1 — Programa de teste

Escreva uma função `main` que: cria uma pilha, empilha os valores 3, 7, 1, 9, imprime o tamanho, desempilha e imprime todos os elementos (um por linha) e, por fim, destrói a pilha. Confirme na saída que a ordem de remoção é 9, 1, 7, 3 (o último a entrar é o primeiro a sair).

1.2 — topo (espiar sem remover)

Implemente

```
int topo(struct pilha *p);
```

que devolve o elemento do topo sem desempilhá-lo. A pilha deve continuar exatamente igual depois da chamada. Trate o caso de pilha vazia.

1.3 — esta_vazia e esta_cheia

Implemente duas funções que devolvem 1 (verdadeiro) ou 0 (falso):

```
int esta_vazia(struct pilha *p);
int esta_cheia(struct pilha *p);
```

Depois, reescreva as suas versões de `empilha` e `desempilha` usando essas funções em vez de comparar `topo` com números "na mão".

1.4 — esvazia

Implemente `void esvazia(struct pilha *p);` que remove todos os elementos da pilha (deixando-a vazia) sem destruí-la. Ela deve poder ser usada normalmente depois.

1.5 — Imprimir sem destruir

Implemente `void imprime(struct pilha *p);` que imprime os elementos do topo até o fundo, sem alterar a pilha ao final. Dica: use uma pilha auxiliar.

Nível 2 — Manipulando pilhas

2.1 — Copiar uma pilha

Implemente `struct pilha *copia(struct pilha *p);` que devolve uma nova pilha com os mesmos elementos, na mesma ordem, sem modificar a original.

2.2 — Duas pilhas são iguais?

Implemente `int iguais(struct pilha *a, struct pilha *b)`; que devolve verdadeiro se as duas pilhas têm os mesmos elementos, na mesma ordem. Ao final, `a` e `b` devem permanecer intactas.

2.3 — Inverter uma pilha

Implemente `void inverte(struct pilha *p)`; que inverte a ordem dos elementos da pilha (o que estava no fundo passa para o topo). Use apenas pilhas auxiliares — nada de mexer no vetor diretamente.

2.4 — Inverter uma string

Leia uma palavra do teclado e use uma pilha para imprimi-la ao contrário. Ex.: `pilha` → `ahlip`.

2.5 — Ordenar uma pilha

Implemente `void ordena(struct pilha *p)`; que ordena os elementos de forma crescente (menor no topo) usando apenas uma pilha auxiliar e as operações de pilha. Não use vetor extra nem funções de ordenação prontas.

Nível 3 — Aplicações clássicas

3.1 — Parênteses balanceados

Escreva `int balanceada(const char *s)`; que recebe uma expressão contendo `(, ), [, ], {, }` e devolve verdadeiro se todos os símbolos de abertura têm o fechamento correto e na ordem certa.

- `"(a+[b*c])"` → verdadeiro
- `"(a+[b])"` → falso
- `")("` → falso

3.2 — Palíndromo

Usando uma pilha, escreva `int palindromo(const char *s)`; que verifica se uma sequência de caracteres é a mesma lida de trás para frente (ex.: `"arara"`).

3.3 — Decimal para binário

Escreva `void decimal_para_binario(int n)`; que imprime a representação binária de `n` usando uma pilha para inverter a ordem dos restos das divisões por 2. Generalize depois para uma base `b` qualquer (2 a 16).

3.4 — Avaliação de expressão pós-fixada (notação da HP-12c)

Escreva `int avalia_posfixa(const char *expr)`; que avalia uma expressão em notação pós-fixada (polonesa reversa), com operandos de um dígito e os operadores `+ - * /` separados por espaço.

- `"3 4 +"` → 7
- `"5 1 2 + 4 * + 3 -"` → 14

Regra: ao ler um número, empilhe; ao ler um operador, desempilhe dois operandos, aplique e empilhe o resultado. No fim, o topo é a resposta.

3.5 — Verificar sequência de push/pop

Você recebe uma sequência de comandos, por exemplo `"E5 E3 D E7 D D"` (onde `E5` = empilha 5 e `D` = desempilha). Escreva um programa que executa a sequência sobre uma pilha e, para cada `D`, imprime o valor removido. Detecte e reporte tentativas de desempilhar com a pilha vazia.

Nível 4 — Desafios

4.1 — Pilha do mínimo (min-stack)

Projete uma pilha que, além de empilhar e desempilhar, ofereça

```
int minimo(struct pilha_min *p);
```

que devolve o menor elemento presente na pilha em tempo. Dica: mantenha uma segunda pilha com os mínimos.

4.2 — Infixa para pós-fixada

Escreva `void infix_a_para_posfixa(const char *in, char *out);` que converte uma expressão infix (ex.: `"3+4*2"`) para pós-fixada (`"3 4 2 * +"`), respeitando a precedência de `+` `-` `*` `/` e os parênteses. Use uma pilha de operadores (algoritmo shunting-yard). Combine com o exercício 3.4 para avaliar a expressão original.

Para todos os exercícios

Para cada função que você implementar, responda:

- Ela funciona corretamente quando a pilha está vazia?
- Ela funciona quando a pilha está cheia (na versão com arranjo)?
- Há algum vazamento de memória (`malloc` sem `free`)?